Decision Quotient: Foundations and Complexity of Decision-Relevant Information

Tristan Simas
McGill University
tristan.simas@mail.mcgill.ca

March 4, 2026

Abstract

We study *decision-relevant information*: which coordinates of a state space suffice to determine the optimal action. For a decision problem $\mathcal{D} = (A, S, U)$ with $S = X_1 \times \cdots \times X_n$, a coordinate set I is sufficient if $s_I = s'_I$ implies $\mathrm{Opt}(s) = \mathrm{Opt}(s')$. The associated optimizer quotient records the coarsest abstraction that preserves optimal-action distinctions.

Under the encoding model fixed in Section 2.4, we prove that Sufficiency-Check is coNP-complete, Minimum-Sufficient-Set is coNP-complete, and Anchor-Sufficiency is Σ_2^{P}-complete. We also prove an encoding-sensitive dichotomy: explicit-state instances with logarithmic-size sufficient sets admit polynomial-time algorithms, while succinctly encoded instances with linear-size sufficient sets inherit ETH-conditioned exponential lower bounds. In addition, we isolate tractable subcases for bounded actions, separable utility, low tensor rank, tree structure, bounded treewidth, and coordinate symmetry.

These results also yield direct corollaries for exact simplification tasks: in the general succinct regime, exact behavior-preserving minimization has no polynomial-time worst-case solution unless $\mathsf{P} = \mathsf{coNP}$, while conservative over-specification can be a rational response when exact pruning carries exponential cost.

The Lean 4 development mechanizes the core reductions and theorem statements, with 44497 lines, 1872 theorems/lemmas, and 179 proof files.

1 Introduction

Which coordinates of a system's state determine the optimal action? For a decision problem $\mathcal{D} = (A, S, U)$ with $S = X_1 \times \cdots \times X_n$, a coordinate set I is sufficient when agreement on I forces agreement on the optimal-action set:

$$s_I = s'_I \implies \mathrm{Opt}(s) = \mathrm{Opt}(s').$$

This is the basic problem of decision-relevant information: how much of the state must remain visible in order to preserve the decision boundary.

The question is easy to state but not algorithmically benign. Evaluating $U(a, s)$ on a given state is one task; certifying that an entire family of coordinates may be ignored is another. The latter carries universal quantification over state pairs and leads naturally to coNP and Σ_2^P phenomena.

© 2026 Tristan Simas. This work is licensed under CC BY 4.0. License: https://creativecommons.org/licenses/by/4.0/

The optimizer quotient packages the same issue structurally: it is the coarsest abstraction of the state space that preserves optimal-action distinctions.

This paper isolates that abstract problem. We work entirely with finite coordinate-structured decision problems and explicit encoding regimes.

Section 2.4 fixes the computational model and the encoding regimes used by every complexity claim.

1.1 Contributions

1. **Shared foundations for coordinate sufficiency.** We formalize decision problems with coordinate structure, relevant-coordinate witnesses, exact relevance identification, and the optimizer quotient as the canonical decision-preserving abstraction.

2. **Exact complexity classifications.** We prove that Sufficiency-Check is coNP-complete, Minimum-Sufficient-Set is coNP-complete, and Anchor-Sufficiency is Σ_2^P-complete.

3. **Encoding-sensitive hardness and tractability.** We prove an explicit-state versus succinct dichotomy, derive ETH-conditioned lower bounds, and isolate natural tractable subcases under structural restrictions.

4. **Direct engineering corollaries.** We show that exact behavior-preserving minimization inherits the hardness of Minimum-Sufficient-Set, while conservative over-specification can be rational in the hard succinct regime under an explicit cost model.

5. **Machine-checked reductions and proof artifacts.** The main reductions and supporting lemmas are mechanized in Lean 4, yielding a reproducible proof artifact for the core complexity claims.

1.2 Why the Problem Is Different

The slogan of the paper is simple:

> **Determining what you need to know can be harder than evaluating the full objective once everything is known.**

The complexity gap comes from the structure of the question. Insufficiency has short witnesses: two states that agree on the proposed coordinates but induce different optimal-action sets. Sufficiency, by contrast, asks for the absence of every such witness. This difference drives the coNP classification and explains why relevance identification is not just another forward evaluation problem.

1.3 Formalization Snapshot

Metric	Value
Lines of Lean 4	44497
Theorems/lemmas	1872
Proof files	179
`sorry` count	0

The artifact builds with `lake build`. The purpose of the mechanization here is not to replace the mathematical narrative, but to guarantee the correctness of the reduction constructions and the formal statements attached to them.

1.4 Paper Structure

Section 2 introduces the abstract foundations and the encoding contract. The hardness theorems are stated in Section 3. Section 4 gives the encoding-sensitive dichotomy and ETH chain, Section 5 records tractable subcases, and Section 6 states the resulting engineering corollaries. Section 7 situates the work within formalized complexity theory and decision-theoretic relevance. Section A summarizes the Lean artifact.

1.5 Artifact Availability

The complete Lean 4 formalization is available at:

https://doi.org/10.5281/zenodo.18140965

The proofs build with the Lean toolchain specified in `lean-toolchain`.

2 Formal Foundations

This section fixes the abstract, non-physical vocabulary used throughout the paper. We work with finite decision problems whose state spaces factor into coordinates, and we isolate the minimal terminology needed to state the later complexity results.

2.1 Decision Problems with Coordinate Structure

Definition 2.1 (Decision Problem)**.** A *decision problem with coordinate structure* is a tuple $\mathcal{D} = (A, X_1, \ldots, X_n, U)$ where:

- A is a finite set of *actions* (alternatives)
- $X_1, \ldots, X_n$ are finite *coordinate spaces*
- $S = X_1 \times \cdots \times X_n$ is the *state space*
- $U : A \times S \to \mathbb{Q}$ is the *utility function*

Definition 2.2 (Projection)**.** For state $s = (s_1, \ldots, s_n) \in S$ and coordinate set $I \subseteq \{1, \ldots, n\}$:

$$s_I := (s_i)_{i \in I}$$

is the *projection* of s onto coordinates in I.

Definition 2.3 (Optimizer Map)**.** For state $s \in S$, the *optimal action set* is:

$$\mathrm{Opt}(s) := \arg\max_{a \in A} U(a, s) = \{a \in A : U(a, s) = \max_{a' \in A} U(a', s)\}$$

2.2 Sufficiency and Relevance

Definition 2.4 (Sufficient Coordinate Set)**.** A coordinate set $I \subseteq \{1, \ldots, n\}$ is *sufficient* for decision problem $\mathcal{D}$ if:

$$\forall s, s' \in S : \quad s_I = s'_I \implies \mathrm{Opt}(s) = \mathrm{Opt}(s')$$

Equivalently, the optimal action depends only on coordinates in I.

Proposition 2.5 (Empty-Set Sufficiency Equals Constant Decision Boundary)**.** *The empty set $\emptyset$ is sufficient if and only if Opt(s) is constant over the entire state space.*

Proof. If $\emptyset$ is sufficient, then every two states agree on their empty projection, so sufficiency forces $\mathrm{Opt}(s) = \mathrm{Opt}(s')$ for all $s, s' \in S$. The converse is immediate. ■

Proposition 2.6 (Insufficiency Equals Counterexample Witness)**.** *Coordinate set I is not sufficient if and only if there exist states $s, s' \in S$ such that $s_I = s'_I$ but Opt$(s) \neq$ Opt(s').*

Proof. This is the direct negation of Definition 2.4. ■

Definition 2.7 (Minimal Sufficient Set)**.** A sufficient set I is *minimal* if no proper subset $I' \subsetneq I$ is sufficient.

Definition 2.8 (Relevant Coordinate)**.** Coordinate i is *relevant* if there exist states that differ only at coordinate i and induce different optimal-action sets:

$$i \text{ is relevant} \iff \exists s, s' \in S : \left(\forall j \neq i,\ s_j = s'_j\right) \ \wedge\ \mathrm{Opt}(s) \neq \mathrm{Opt}(s').$$

Proposition 2.9 (Minimal-Set/Relevance Equivalence)**.** *For any minimal sufficient set I and any coordinate i,*

$$i \in I \iff i \text{ is relevant.}$$

Hence every minimal sufficient set is exactly the relevant-coordinate set.

Proof. If $i \in I$ were not relevant, then changing coordinate i while keeping all others fixed would never alter the optimal-action set, so removing i would preserve sufficiency, contradicting minimality. Conversely, every sufficient set must contain each relevant coordinate, because omitting a relevant coordinate leaves a witness pair with identical I-projection but different optimal-action sets. ■

Definition 2.10 (Exact Relevance Identifiability)**.** For a decision problem $\mathcal{D}$ and candidate set I, we say that I is *exactly relevance-identifying* if

$$\forall i \in \{1, \ldots, n\} : \quad i \in I \iff i \text{ is relevant for } \mathcal{D}.$$

2.3 Optimizer Quotient

Definition 2.11 (Decision Equivalence)**.** States $s, s' \in S$ are *decision-equivalent* if they induce the same optimal-action set:

$$s \sim_{\mathrm{Opt}} s' \iff \mathrm{Opt}(s) = \mathrm{Opt}(s').$$

Definition 2.12 (Decision Quotient)**.** The *decision quotient* (equivalently, optimizer quotient) of $\mathcal{D}$ is the quotient object

$$Q_{\mathcal{D}} := S/{\sim_{\mathrm{Opt}}}.$$

Its equivalence classes are exactly the maximal subsets of states on which the optimal-action set is constant.

Proposition 2.13 (Sufficiency Characterization)**.** *Coordinate set I is sufficient if and only if the optimizer map factors through the projection $\pi_I : S \to S_I$. Equivalently, there exists a function $\overline{Opt}_I : S_I \to \mathcal{P}(A)$ such that*

$$Opt = \overline{Opt}_I \circ \pi_I.$$

Proof. If I is sufficient, define $\overline{\mathrm{Opt}}_I(x)$ to be $\mathrm{Opt}(s)$ for any state s with $s_I = x$; this is well defined exactly because sufficiency makes Opt constant on each projection fiber. Conversely, any such factorization forces states with equal I-projection to have equal optimal-action sets. ■

2.4 Computational Model and Input Encoding

The same decision question yields different computational problems under different input models. We therefore fix the encoding regime before stating any complexity classification.

Succinct encoding (primary for hardness). An instance is encoded by:

- a finite action set A given explicitly,
- coordinate domains $X_1, \ldots, X_n$ given by their sizes in binary,
- a Boolean or arithmetic circuit C_U that on input (a, s) outputs $U(a, s)$.

The input length is

$$L = |A| + \sum_i \log |X_i| + |C_U|.$$

Unless stated otherwise, all class-theoretic hardness results in this paper are measured in L.

Explicit-state encoding. The utility function is given as a full table over $A \times S$. The input length is

$$L_{\text{exp}} = \Theta(|A||S|)$$

up to the bitlength of the utility entries. Polynomial-time claims stated in terms of $|S|$ use this explicit-state regime.

Query-access regime. The solver is given oracle access to $\text{Opt}(s)$ or to the utility values needed to reconstruct it. Complexity is then measured by query count, optionally paired with per-query evaluation cost. This regime separates access obstruction from full-table availability and from succinct circuit input length.

Remark 2.14 (Decision Questions vs Decision Problems). The predicate "is coordinate set I sufficient for $\mathcal{D}$?" is an encoding-independent mathematical question. A *decision problem* arises only after fixing how $\mathcal{D}$ and I are represented. Complexity classes are properties of these encoded problems, not of the abstract predicate in isolation.

3 Computational Complexity of Decision-Relevant Uncertainty

This section establishes the computational complexity of determining which state coordinates are decision-relevant, under the succinct encoding of Section 2.4. We prove three main results:

1. **SUFFICIENCY-CHECK** is coNP-complete
2. **MINIMUM-SUFFICIENT-SET** is coNP-complete (the Σ_2^P structure collapses)
3. **ANCHOR-SUFFICIENCY** (fixed coordinates) is Σ_2^P-complete

These results sit beyond NP-completeness and explain why engineers default to over-modeling: finding the minimal set of decision-relevant factors is computationally intractable.

3.1 Problem Definitions

Definition 3.1 (Decision Problem Encoding)**.** A *decision problem instance* is a tuple (A, n, U) where:

- A is a finite set of alternatives
- n is the number of state coordinates, with state space $S = \{0, 1\}^n$
- $U : A \times S \to \mathbb{Q}$ is the utility function, given as a Boolean circuit

Definition 3.2 (Optimizer Map)**.** For state $s \in S$, define:

$$\mathrm{Opt}(s) := \arg\max_{a \in A} U(a, s)$$

Definition 3.3 (Sufficient Coordinate Set)**.** A coordinate set $I \subseteq \{1, \ldots, n\}$ is *sufficient* if:

$$\forall s, s' \in S : \quad s_I = s'_I \implies \mathrm{Opt}(s) = \mathrm{Opt}(s')$$

where s_I denotes the projection of s onto coordinates in I.

Problem 3.4 (SUFFICIENCY-CHECK)**. Input:** Decision problem (A, n, U) and coordinate set $I \subseteq \{1, \ldots, n\}$
Question: Is I sufficient?

Problem 3.5 (MINIMUM-SUFFICIENT-SET)**. Input:** Decision problem (A, n, U) and integer k
Question: Does there exist a sufficient set I with $|I| \leq k$?

3.2 Hardness of SUFFICIENCY-CHECK

Theorem 3.6 (coNP-completeness of SUFFICIENCY-CHECK)**.** *SUFFICIENCY-CHECK is coNP-complete [10, 19].* (Machine-verified in Lean 4; see `Hardness/CoNPComplete.lean`.)

Source	Target	Key property preserved
TAUTOLOGY	SUFFICIENCY-CHECK	Tautology iff $\emptyset$ sufficient
$\exists\forall$-SAT	ANCHOR-SUFFICIENCY	Witness anchors iff formula true
SET-COVER	MINIMUM-SUFFICIENT-SET	Set size maps to coordinate size

Proof. **Membership in coNP:** The complementary problem INSUFFICIENCY is in NP. Given (A, n, U, I), a witness for insufficiency is a pair (s, s') such that:

1. $s_I = s'_I$ (verifiable in polynomial time)
2. $\mathrm{Opt}(s) \neq \mathrm{Opt}(s')$ (verifiable by evaluating U on all alternatives)

coNP-hardness: We reduce from TAUTOLOGY.
Given Boolean formula $\varphi(x_1, \ldots, x_n)$, construct a decision problem with:

- Alternatives: $A = \{\text{accept}, \text{reject}\}$
- State space: $S = \{\text{reference}\} \cup \{0, 1\}^n$

- Utility:

$$\begin{aligned} U(\text{accept}, \text{reference}) &= 1 \\ U(\text{reject}, \text{reference}) &= 0 \\ U(\text{accept}, a) &= \varphi(a) \\ U(\text{reject}, a) &= 0 \quad \text{for assignments } a \in \{0,1\}^n \end{aligned}$$

- Query set: $I = \emptyset$

Claim: $I = \emptyset$ is sufficient $\iff$ φ is a tautology.

($\Rightarrow$) Suppose I is sufficient. Then $\text{Opt}(s)$ is constant over all states. Since $U(\text{accept}, a) = \varphi(a)$ and $U(\text{reject}, a) = 0$:

- $\text{Opt}(a) = \text{accept}$ when $\varphi(a) = 1$
- $\text{Opt}(a) = \{\text{accept}, \text{reject}\}$ when $\varphi(a) = 0$

For Opt to be constant, $\varphi(a)$ must be true for all assignments a; hence φ is a tautology.

($\Leftarrow$) If φ is a tautology, then $U(\text{accept}, a) = 1 > 0 = U(\text{reject}, a)$ for all assignments a. Thus $\text{Opt}(s) = \{\text{accept}\}$ for all states s, making $I = \emptyset$ sufficient. ■

3.3 Complexity of MINIMUM-SUFFICIENT-SET

Theorem 3.7 (MINIMUM-SUFFICIENT-SET is coNP-complete)**.** *MINIMUM-SUFFICIENT-SET is coNP-complete.* (Machine-verified.)

Proof. **Structural observation:** The $\exists\forall$ quantifier pattern suggests Σ_2^P:

$$\exists I\, (|I| \leq k)\, \forall s, s' \in S: \quad s_I = s'_I \implies \text{Opt}(s) = \text{Opt}(s')$$

However, this collapses because sufficiency has a simple characterization.

Key lemma: A coordinate set I is sufficient if and only if I contains all relevant coordinates (proven formally as `sufficient_contains_relevant` in Lean):

$$\text{sufficient}(I) \iff \text{Relevant} \subseteq I$$

where $\text{Relevant} = \{i : \exists s, s'.\ s \text{ differs from } s' \text{ only at } i \text{ and } \text{Opt}(s) \neq \text{Opt}(s')\}$.

Consequence: The minimum sufficient set is exactly the set of relevant coordinates. Thus MINIMUM-SUFFICIENT-SET asks: "Is the number of relevant coordinates at most k?"

coNP membership: A witness that the answer is NO is a set of $k+1$ coordinates, each proven relevant (by exhibiting s, s' pairs). Verification is polynomial.

coNP-hardness: The $k = 0$ case asks whether no coordinates are relevant, i.e., whether $\emptyset$ is sufficient. This is exactly SUFFICIENCY-CHECK, which is coNP-complete by Theorem 3.6. ■

3.4 Anchor Sufficiency (Fixed Coordinates)

We also formalize a strengthened variant that fixes the coordinate set and asks whether there exists an *assignment* to those coordinates that makes the optimal action constant on the induced subcube.

Problem 3.8 (ANCHOR-SUFFICIENCY)**. Input:** Decision problem (A, n, U) and fixed coordinate set $I \subseteq \{1, \ldots, n\}$
Question: Does there exist an assignment α to I such that $\text{Opt}(s)$ is constant for all states s with $s_I = \alpha$?

Theorem 3.9 (ANCHOR-SUFFICIENCY is Σ_2^P-complete)**.** *ANCHOR-SUFFICIENCY is Σ_2^P-complete [36] (already for Boolean coordinate spaces).* (Machine-verified.)

Proof. **Membership in Σ_2^P:** The problem has the form

$$\exists \alpha\, \forall s \in S :\ (s_I = \alpha) \implies \mathrm{Opt}(s) = \mathrm{Opt}(s_\alpha),$$

which is an $\exists\forall$ pattern.

Σ_2^P-hardness: Reduce from $\exists\forall$-SAT. Given $\exists x\, \forall y\, \varphi(x, y)$ with $x \in \{0,1\}^k$ and $y \in \{0,1\}^m$, if $m = 0$ we first pad with a dummy universal variable (satisfiability is preserved), construct a decision problem with:

- Actions $A = \{\mathrm{YES}, \mathrm{NO}\}$
- State space $S = \{0,1\}^{k+m}$ representing (x, y)
- Utility
$$U(\mathrm{YES}, (x,y)) = \begin{cases} 2 & \text{if } \varphi(x,y) = 1 \\ 0 & \text{otherwise} \end{cases} \qquad U(\mathrm{NO}, (x,y)) = \begin{cases} 1 & \text{if } y = 0^m \\ 0 & \text{otherwise} \end{cases}$$
- Fixed coordinate set $I =$ the x-coordinates.

If $\exists x^\star\, \forall y\, \varphi(x^\star, y) = 1$, then for any y we have $U(\mathrm{YES}) = 2$ and $U(\mathrm{NO}) \le 1$, so $\mathrm{Opt}(x^\star, y) = \{\mathrm{YES}\}$ is constant. Conversely, if $\varphi(x, y)$ is false for some y, then either $y = 0^m$ (where NO is optimal) or $y \neq 0^m$ (where YES and NO tie), so the optimal set varies across y and the subcube is not constant. Thus an anchor assignment exists iff the $\exists\forall$-SAT instance is true. ■

3.5 Tractable Subcases

Despite the general hardness, several natural subcases admit efficient algorithms:

Proposition 3.10 (Small State Space)**.** *When $|S|$ is polynomial in the input size (i.e., explicitly enumerable), MINIMUM-SUFFICIENT-SET is solvable in polynomial time.*

Proof. Compute $\mathrm{Opt}(s)$ for all $s \in S$. The minimum sufficient set is exactly the set of coordinates that "matter" for the resulting function, computable by standard techniques. ■

Proposition 3.11 (Linear Utility)**.** *When $U(a, s) = w_a \cdot s$ for weight vectors $w_a \in \mathbb{Q}^n$, MINIMUM-SUFFICIENT-SET reduces to identifying coordinates where weight vectors differ.*

3.6 Implications

Corollary 3.12 (Why Over-Modeling Is Rational)**.** *Finding the minimal set of decision-relevant factors is coNP-complete. Even* verifying *that a proposed set is sufficient is coNP-complete.*

This formally explains the engineering phenomenon:

1. *It's computationally easier to model everything than to find the minimum*
2. *"Which unknowns matter?" is a hard question, not a lazy one to avoid*
3. *Bounded scenario analysis (small $\hat{S}$) makes the problem tractable*

This connects to the pentalogy's leverage framework: the "epistemic budget" for deciding what to model is itself a computationally constrained resource.

3.7 Remark: The Collapse to coNP

Early analysis of MINIMUM-SUFFICIENT-SET focused on the apparent $\exists\forall$ quantifier structure, which suggested a Σ_2^P-complete result. We initially explored certificate-size lower bounds for the complement, attempting to show MINIMUM-SUFFICIENT-SET was unlikely to be in coNP.

However, the key insight—formalized as `sufficient_contains_relevant`—is that sufficiency has a simple characterization: a set is sufficient iff it contains all relevant coordinates. This collapses the $\exists\forall$ structure because:

- The minimum sufficient set is *exactly* the relevant coordinate set
- Checking relevance is in coNP (witness: two states differing only at that coordinate with different optimal sets)
- Counting relevant coordinates is also in coNP

This collapse explains why ANCHOR-SUFFICIENCY retains its Σ_2^P-completeness: fixing coordinates and asking for an assignment that works is a genuinely different question. The "for all suffixes" quantifier cannot be collapsed when the anchor assignment is part of the existential choice.

4 Complexity Dichotomy

The hardness results of Section 3 apply to worst-case instances under the succinct encoding. This section states a dichotomy that separates an explicit-state upper bound from a succinct-encoding lower bound. The dichotomy and ETH reduction chain are machine-verified in Lean 4.

Model note. Part 1 is an explicit-state upper bound (time polynomial in $|S|$). Part 2 is a succinct-encoding lower bound under ETH (time exponential in n). The encodings are defined in Section 2.4.

Theorem 4.1 (Complexity Dichotomy)**.** *Let $\mathcal{D} = (A, X_1, \ldots, X_n, U)$ be a decision problem with $|S| = N$ states. Let k^* be the size of the minimal sufficient set.*

1. ***Logarithmic case (explicit-state upper bound):*** *If $k^* = O(\log N)$, then SUFFICIENCY-CHECK is solvable in polynomial time in N under the explicit-state encoding.*
2. ***Linear case (succinct lower bound under ETH):*** *If $k^* = \Omega(n)$, then SUFFICIENCY-CHECK requires time $\Omega(2^{n/c})$ for some constant $c > 0$ under the succinct encoding (assuming ETH).*

Proof. **Part 1 (Logarithmic case):** If $k^* = O(\log N)$, then the number of distinct projections $|S_{I^*}|$ is at most $2^{k^*} = O(N^c)$ for some constant c. Under the explicit-state encoding, we enumerate all projections and verify sufficiency in polynomial time.

Part 2 (Linear case): We establish this via an explicit reduction chain from the Exponential Time Hypothesis under the succinct encoding. ■

4.1 The ETH Reduction Chain

The lower bound in Part 2 of Theorem 4.1 follows from a chain of reductions originating in the Exponential Time Hypothesis. We make this chain explicit.

Definition 4.2 (Exponential Time Hypothesis (ETH))**.** There exists a constant $\delta > 0$ such that 3-SAT on n variables cannot be solved in time $O(2^{\delta n})$ [18].

The chain proceeds as follows:

1. **ETH $\Rightarrow$ 3-SAT requires $2^{\Omega(n)}$:** This is the definition of ETH.
2. **3-SAT $\leq_p$ TAUTOLOGY:** Given 3-SAT formula $\varphi(x_1, \ldots, x_n)$, define $\psi = \neg\varphi$. Then φ is satisfiable iff ψ is not a tautology. This is a linear-time reduction preserving the number of variables.
3. **TAUTOLOGY requires $2^{\Omega(n)}$ (under ETH):** By the contrapositive of step 2, if TAUTOLOGY is solvable in $o(2^{\delta n})$ time, then 3-SAT is solvable in $o(2^{\delta n})$ time, contradicting ETH.
4. **TAUTOLOGY $\leq_p$ SUFFICIENCY-CHECK:** This is Theorem 3.6. Given formula $\varphi(x_1, \ldots, x_n)$, we construct a decision problem where:
   - The empty set $I = \emptyset$ is sufficient iff φ is a tautology
   - When φ is not a tautology, all n coordinates are relevant

   The reduction is polynomial-time and preserves the number of coordinates.
5. **SUFFICIENCY-CHECK requires $2^{\Omega(n)}$ (under ETH):** Combining steps 3 and 4: if SUFFICIENCY-CHECK is solvable in $o(2^{\delta n/c})$ time for some constant c, then TAUTOLOGY (and hence 3-SAT) is solvable in subexponential time, contradicting ETH.

Proposition 4.3 (Tight Constant)**.** *The reduction in Theorem 3.6 preserves the number of variables up to an additive constant: an n-variable formula yields an $(n+1)$-coordinate decision problem. Therefore, the constant c in the $2^{n/c}$ lower bound is asymptotically 1:*

$$\textit{SUFFICIENCY-CHECK requires time } \Omega(2^{\delta(n-1)}) = 2^{\Omega(n)} \textit{ under ETH}$$

where δ is the ETH constant for 3-SAT.

Proof. The TAUTOLOGY reduction (Theorem 3.6) constructs:

- State space $S = \{\text{ref}\} \cup \{0,1\}^n$ with $n+1$ coordinates (one extra for the reference state)
- Query set $I = \emptyset$

When φ has n variables, the constructed problem has $n+1$ coordinates. The asymptotic lower bound is $2^{\Omega(n)}$ with the same constant δ from ETH. ■

4.2 Phase Transition

Corollary 4.4 (Phase Transition)**.** *There is a sharp transition between tractable and intractable regimes at the logarithmic scale (with respect to the encodings in Section 2.4):*

- *If $k^* = O(\log N)$, SUFFICIENCY-CHECK is polynomial in N under the explicit-state encoding*
- *If $k^* = \Omega(n)$, SUFFICIENCY-CHECK is exponential in n under ETH in the succinct encoding*

For Boolean coordinate spaces ($N = 2^n$), the transition is between $k^* = O(\log n)$ (explicit-state tractable) and $k^* = \Omega(n)$ (succinct-encoding intractable).

Proof. The logarithmic case (Part 1 of Theorem 4.1) gives polynomial time when $k^* = O(\log N)$. More precisely, when $k^* \leq c \log N$ for constant c, the algorithm runs in time $O(N^c \cdot \text{poly}(n))$.

The linear case (Part 2) gives exponential time when $k^* = \Omega(n)$.

The transition point is where $2^{k^*} = N^{k^*/\log N}$ stops being polynomial in N, i.e., when $k^* = \omega(\log N)$. For Boolean coordinate spaces ($N = 2^n$), this corresponds to the gap between $k^* = O(\log n)$ and $k^* = \Omega(n)$. ■

Remark 4.5 (Sharpness of Dichotomy). Under ETH, the lower bound is asymptotically tight in the succinct encoding. The explicit-state upper bound is tight in the sense that it matches enumeration complexity in N.

This dichotomy explains why some domains admit tractable model selection (few relevant variables) while others require heuristics (many relevant variables). In the succinct regime, the reduction chain gives a class-level coNP statement together with a worst-case $2^{\Omega(n)}$ lower bound under the declared ETH assumption.

Bridge theorems. The ETH chain composes with structural rank characterizations to yield the complete complexity picture. Specifically: `BR1` combines ETH hardness with structural rank to show that problems with maximal srank (all coordinates relevant) inherit the exponential lower bound; `DQ1` composes the dichotomy theorem with ETH to give the full classification; `DQ3` combines the TAUTOLOGY reduction with ETH to establish coNP-completeness plus exponential hardness. These bridge theorems connect the complexity cluster (ETH, reductions) to the geometric cluster (srank, sufficiency) in the proof graph.

Remark 4.6 (Circuit Model Formalization). The ETH lower bound is stated in the succinct circuit encoding of Section 2.4, where the utility function $U : A \times S \to \mathbb{R}$ is represented by a Boolean circuit computing $\mathbf{1}[U(a, s) > \theta]$ for threshold comparisons. In this model:

- The input size is the circuit size m, not the state space size $|S| = 2^n$
- A 3-SAT formula with n variables and c clauses yields a circuit of size $O(n + c)$
- The reduction preserves instance size up to constant factors: $m_{\text{out}} \leq 3 \cdot m_{\text{in}}$

This linear size preservation is essential for ETH transfer. In the explicit enumeration model (where S is given as a list), the reduction would blow up the instance size exponentially, precluding ETH-based lower bounds. The circuit model is standard in fine-grained complexity and matches practical representations of decision problems.

5 Tractable Special Cases

We distinguish the encodings of Section 2.4. The tractability results below state the model assumption explicitly. All results are formalized in `DecisionQuotient/Tractability/` and verified in Lean 4 with no `sorry` placeholders.

Theorem 5.1 (Tractable Subcases)**.** *SUFFICIENCY-CHECK is polynomial-time solvable in the following cases:*

1. ***Bounded actions (explicit-state encoding):*** *The input contains the full utility table over $A \times S$ with $|A| \leq k$ for constant k. Complexity: $O(|S|^2 \cdot k^2)$.*

2. ***Separable utility (rank 1):*** *$U(a, s) = f(a) + g(s)$. Complexity: $O(1)$ (trivially sufficient).*

3. ***Low tensor rank:*** *The utility admits a rank-R decomposition $U(a, s) = \sum_{r=1}^{R} w_r \cdot f_r(a) \cdot \prod_i g_{ri}(s_i)$. Complexity: $O(|A| \cdot R \cdot n)$.*

4. ***Tree-structured dependencies:*** *Coordinates are ordered such that s_i depends only on $(s_1, \ldots, s_{i-1})$. Complexity: $O(n \cdot |A| \cdot k_{\max})$ where $k_{\max} = \max_i |X_i|$.*

5. ***Bounded treewidth:*** *The interaction graph has treewidth $\leq w$ and utility factors over edges. Complexity: $O(n \cdot k^{w+1})$ via standard CSP algorithms.*

6. ***Coordinate symmetry:*** *Utility is invariant under coordinate permutations. Complexity: $O\left(\binom{d+k-1}{k-1}^2\right)$ orbit-type pairs.*

The following subsections provide formal definitions, reduction theorems, and prose descriptions for each tractable subcase. For each result, we cite standard algorithms where applicable and prove only the paper-specific reductions in Lean.

5.1 Bounded Actions

When the action set is bounded, brute-force enumeration becomes polynomial.

Definition 5.2 (Bounded Action Problem)**.** A decision problem has *bounded actions* with parameter k if $|A| \leq k$ for some constant k independent of the state space size.

Theorem 5.3 (Bounded Actions Tractability)**.** *If $|A| \leq k$ for constant k, then SUFFICIENCY-CHECK runs in $O(|S|^2 \cdot k^2)$ time.*

Proof. Given the full table of $U(a, s)$:

1. Compute $\mathrm{Opt}(s)$ for all $s \in S$ in $O(|S| \cdot k)$ time.
2. For each pair (s, s') with $s_I = s'_I$, compare $\mathrm{Opt}(s)$ and $\mathrm{Opt}(s')$ in $O(k^2)$ time.
3. Total: $O(|S|^2)$ pairs $\times$ $O(k^2)$ comparisons $= O(|S|^2 \cdot k^2)$.

When k is constant, this is polynomial in $|S|$. ■

Lean formalization: `DecisionQuotient/Tractability/BoundedActions.lean`

- `L9`: Correctness of the algorithm.
- `L1`: Complexity bound $|S|^2 \cdot (1 + k^2)$.

5.2 Separable Utility (Rank 1)

Separable utility decouples actions from states, making sufficiency trivial.

Definition 5.4 (Separable Utility)**.** A utility function is *separable* if there exist functions $f : A \to \mathbb{R}$ and $g : S \to \mathbb{R}$ such that $U(a, s) = f(a) + g(s)$ for all $(a, s) \in A \times S$.

Theorem 5.5 (Separable Tractability)**.** *If utility is separable, then $I = \emptyset$ is always sufficient.*

Proof. If $U(a, s) = f(a) + g(s)$, then:

$$\mathrm{Opt}(s) = \arg\max_{a \in A}[f(a) + g(s)] = \arg\max_{a \in A} f(a)$$

The optimal action depends only on f, not on s. Hence $\mathrm{Opt}(s) = \mathrm{Opt}(s')$ for all s, s', making any I (including $\emptyset$) sufficient. ■

Lean formalization: `DecisionQuotient/Tractability/SeparableUtility.lean`

- `SeparableUtility`: Structure encoding the separability condition.
- `L8`: Optimal actions are state-independent.
- `L7`: The empty set is sufficient.

5.3 Low Tensor Rank

Utility functions with low tensor rank admit efficient factored computation, generalizing separable utility.

Definition 5.6 (Tensor Rank Decomposition)**.** A utility function $U : A \times ((i : \mathsf{Fin}\ n) \to X_i) \to \mathbb{R}$ has *tensor rank* R if there exist weights $w_1, \ldots, w_R \in \mathbb{R}$, action factors $f_r : A \to \mathbb{R}$, and coordinate factors $g_{ri} : X_i \to \mathbb{R}$ such that:

$$U(a, s) = \sum_{r=1}^{R} w_r \cdot f_r(a) \cdot \prod_{i=1}^{n} g_{ri}(s_i)$$

Remark 5.7. Separable utility (Definition 5.4) is the special case $R = 1$ with $w_1 = 1$, $f_1(a) = f(a)$, and $\prod_i g_{1i}(s_i) = g(s)$.

Theorem 5.8 (Low Tensor Rank Tractability)**.** *If utility has tensor rank R, then SUFFICIENCY-CHECK runs in $O(|A| \cdot R \cdot n)$ time per state.*

Proof. The factored representation enables efficient computation:

1. For each rank component r, the contribution $w_r \cdot f_r(a) \cdot \prod_i g_{ri}(s_i)$ factors over coordinates.
2. Two states s, s' agreeing on I have $g_{ri}(s_i) = g_{ri}(s'_i)$ for $i \in I$.
3. The partial product $\prod_{i \in I} g_{ri}(s_i)$ is identical for agreeing states.
4. This reduces sufficiency checking to comparing factored sums over orbit representatives.

■

Lean formalization: `DecisionQuotient/Tractability/SeparableUtility.lean`

- `TensorRankDecomposition`: Structure encoding rank-R decomposition.
- `L14`: Axiom citing tensor network algorithms [20, 9].
- `L4`: Reduction theorem for factored computation.
- `L5`: Tractability via factored sufficiency check.

5.4 Tree-Structured Dependencies

When coordinate dependencies form a tree, dynamic programming yields polynomial-time sufficiency checking.

Definition 5.9 (Tree-Structured Dependencies)**.** A decision problem has *tree-structured dependencies* if there exists an ordering of coordinates $(1, \ldots, n)$ such that the utility decomposes as:

$$U(a, s) = \sum_{i=1}^{n} u_i(a, s_i, s_{\text{parent}(i)})$$

where $\text{parent}(i) < i$ for all $i > 1$, and the root term depends only on (a, s_1).

Theorem 5.10 (Tree Structure Tractability)**.** *If dependencies are tree-structured with explicit local tables, SUFFICIENCY-CHECK runs in $O(n \cdot |A| \cdot k_{\max})$ time where $k_{\max} = \max_i |X_i|$.*

Proof. Dynamic programming on the tree order:

1. Process coordinates in order $i = n, n-1, \ldots, 1$.
2. For each node i and each value of its parent coordinate, compute the set of actions optimal for some subtree assignment.
3. Combine child summaries with local tables in $O(|A| \cdot k_{\max})$ per node.
4. Total: $O(n \cdot |A| \cdot k_{\max})$.

A coordinate is relevant iff varying its value changes the optimal action set. ■

Lean formalization: `DecisionQuotient/Tractability/TreeStructure.lean`

- `TreeStructured`: Predicate encoding tree dependency structure.
- `L15`: Axiom citing standard DP on trees.

5.5 Bounded Treewidth

The tree-structured case generalizes to bounded treewidth interaction graphs via CSP reduction.

Definition 5.11 (Interaction Graph)**.** Given a pairwise utility decomposition, the *interaction graph* $G = (V, E)$ has:

- Vertices $V = \{1, \ldots, n\}$ (one per coordinate)
- Edges $E = \{(i, j) : i \text{ and } j \text{ interact in the utility}\}$

Definition 5.12 (Pairwise Utility)**.** A utility function is *pairwise* if it decomposes as:

$$U(a, s) = \sum_{i=1}^{n} u_i(a, s_i) + \sum_{(i,j) \in E} u_{ij}(a, s_i, s_j)$$

with unary terms u_i and binary terms u_{ij} given explicitly.

Theorem 5.13 (Bounded Treewidth Tractability)**.** *If the interaction graph has treewidth $\leq w$ and utility is pairwise with explicit factors, then SUFFICIENCY-CHECK runs in $O(n \cdot k^{w+1})$ time where $k = \max_i |X_i|$.*

Proof. We reduce SUFFICIENCY-CHECK to constraint satisfaction on the interaction graph:

1. **Reduction:** For each action $a \in A$, checking whether $\mathrm{Opt}(s) = \mathrm{Opt}(s')$ for states agreeing on I reduces to a CSP on G with variables $\{s_i : i \notin I\}$.
2. **Standard algorithm:** CSP on graphs of treewidth w is solvable in $O(n \cdot k^{w+1})$ via tree decomposition [6, 14].

The reduction is polynomial; the algorithm is standard. ■

Lean formalization: `DecisionQuotient/Tractability/TreeStructure.lean`

- `InteractionGraph`: Definition of the interaction graph.
- `PairwiseUtility`: Structure encoding pairwise decomposition.
- `L16`: Axiom for treewidth (citing Bodlaender [6]).
- `L3`: Axiom citing CSP algorithms.
- `L11`: Paper-specific reduction theorem.
- `L2`: Tractability statement combining reduction and algorithm.

5.6 Coordinate Symmetry

When utility is invariant under coordinate permutations, the effective state space collapses to orbit types.

Definition 5.14 (Dimensional State Space)**.** A *dimensional state space* with alphabet size k and dimension d is $S = \{0, \ldots, k-1\}^d$, the set of d-tuples over a k-element alphabet.

Definition 5.15 (Symmetric Utility)**.** A utility function on a dimensional state space is *symmetric* if it is invariant under coordinate permutations: for all permutations $\sigma \in \mathfrak{S}_d$ and states $s \in S$,

$$U(a, s) = U(a, \sigma \cdot s) \quad \text{where } (\sigma \cdot s)_i = s_{\sigma^{-1}(i)}$$

Definition 5.16 (Orbit Type)**.** The *orbit type* of a state $s \in \{0, \ldots, k-1\}^d$ is the multiset of its coordinate values:

$$\mathrm{orbitType}(s) = \{\{s_1, s_2, \ldots, s_d\}\}$$

Two states have the same orbit type iff they lie in the same orbit under coordinate permutation.

Theorem 5.17 (Orbit Type Characterization)**.** *For dimensional state spaces, two states s, s' have equal orbit types if and only if there exists a coordinate permutation σ such that $\sigma \cdot s = s'$.*

Proof. **If:** Permuting coordinates preserves the multiset of values. **Only if:** If s and s' have the same multiset of values, we can construct a permutation mapping each occurrence in s to the corresponding occurrence in s'. The Lean proof constructs this permutation explicitly via a fiber bundle argument. ■

Theorem 5.18 (Symmetric Sufficiency Reduction)**.** *Under symmetric utility, optimal actions depend only on orbit type:*

$$\mathit{orbitType}(s) = \mathit{orbitType}(s') \implies \mathit{Opt}(s) = \mathit{Opt}(s')$$

Thus SUFFICIENCY-CHECK reduces to checking pairs with different *orbit types.*

Proof. If $\text{orbitType}(s) = \text{orbitType}(s')$, then by Theorem 5.17 there exists σ with $\sigma \cdot s = s'$. By symmetry, $U(a, s) = U(a, s')$ for all a, so $\text{Opt}(s) = \text{Opt}(s')$.

For sufficiency, we need only check pairs (s, s') agreeing on I with *different* orbit types; same-orbit pairs are automatically equal. ■

Theorem 5.19 (Symmetric Complexity Bound)**.** *The number of orbit types is bounded by the stars-and-bars count:*

$$|OrbitTypes| = \binom{d+k-1}{k-1}$$

Thus SUFFICIENCY-CHECK requires at most $\binom{d+k-1}{k-1}^2$ *orbit-type pair checks, which is polynomial in* d *for fixed* k*.*

Proof. An orbit type is a multiset of d values from $\{0, \ldots, k-1\}$, equivalently a k-tuple of non-negative integers summing to d. By stars-and-bars, the count is $\binom{d+k-1}{k-1}$.

For fixed k, this is $O(d^{k-1})$ — polynomial in d. ■

Lean formalization: `DecisionQuotient/Tractability/Dimensional.lean`

- `DimensionalStateSpace`: Definition of k-ary d-dimensional state space.
- `SymmetricUtility`: Predicate for coordinate-permutation invariance.
- `orbitType`: Function computing the orbit type (histogram).
- `L6`: **(Core result)** Orbit types equal iff permutation exists.
- `L12`: Optimal actions constant on orbits.
- `L10`: Reduction to cross-orbit pairs.
- `L13`: $O\left(\binom{d+k-1}{k-1}^2\right)$ bound.

5.7 Practical Implications

The six tractable cases correspond to common modeling scenarios:

- **Bounded actions:** Small action sets (e.g., binary decisions, few alternatives).
- **Separable utility:** Additive cost models, linear utility functions.
- **Low tensor rank:** Factored preference models, low-rank approximations.
- **Tree structure:** Hierarchical decision processes, causal models with tree structure.
- **Bounded treewidth:** Spatial models, graphical models with limited interaction.
- **Coordinate symmetry:** Exchangeable features, order-invariant utilities.

For problems given in the succinct encoding without these structural restrictions, the hardness results of Section 3 apply, justifying heuristic approaches.

5.8 Verification Philosophy

The Lean formalization follows a consistent pattern for each tractable case:

1. **Define the structural restriction** as a Lean structure or predicate.
2. **State axioms for standard algorithms** (DP on trees, CSP on bounded treewidth, tensor contraction) with citations to the literature.
3. **Prove the paper-specific reduction** showing SUFFICIENCY-CHECK satisfies the preconditions of the standard algorithm.
4. **Combine** to obtain the tractability statement.

This separation ensures that:

- Standard results are cited, not re-proved (avoiding redundant mechanization).
- Paper-specific claims are fully verified (the reductions are where errors hide).
- The formalization is maintainable (algorithm updates don't require re-verification).

The philosophy: *mechanize what requires mechanical verification for rational belief; cite what is standard.*

6 Engineering Corollaries

The complexity results of Sections 3–5 have direct engineering corollaries. This section records consequences of the preceding hardness and tractability theorems when the coordinate model is used to represent configuration, feature, or interface choices.

Every claim in this section is model-conditional. The conclusions apply when a design problem is faithfully represented as a decision problem with coordinate structure and when the relevant computational cost is the cost of certifying sufficiency or finding a minimal sufficient set.

6.1 Configuration Simplification as Sufficiency Checking

Proposition 6.1 (Configuration Simplification as Sufficiency-Check)**.** *Let $P = \{p_1, \ldots, p_n\}$ be configuration parameters and let B be a finite set of observable behaviors. Suppose each configuration state determines the subset of behaviors that remain feasible or optimal. Then the question*

"Does parameter subset $I \subseteq P$ preserve all decision-relevant behavior?"

is an instance of Sufficiency-Check.

Proof. Construct a decision problem $\mathcal{D} = (A, X_1, \ldots, X_n, U)$ by taking actions $A = B$, coordinate domains X_i equal to the domains of the parameters p_i, and defining $U(b, s) = 1$ exactly when behavior b is realized or admissible under configuration state s. Then $\mathrm{Opt}(s)$ is the behavior set induced by s, and coordinate subset I is sufficient exactly when agreement on the parameters in I forces agreement on the induced optimal-behavior set. ■

6.2 Limits of Exact Minimization

Corollary 6.2 (No General-Purpose Exact Configuration Minimizer)**.** *Assuming $\mathsf{P} \neq \mathsf{coNP}$, there is no polynomial-time general-purpose procedure that takes an arbitrary succinctly encoded configuration problem and always returns a smallest behavior-preserving parameter subset.*

Proof. Such a procedure would solve Minimum-Sufficient-Set in polynomial time for arbitrary succinctly encoded instances, contradicting Theorem 3.7 under the assumption $\mathsf{P} \neq \mathsf{coNP}$. ■

Remark 6.3. This does not rule out useful tools. It rules out a fully general exact minimizer with worst-case polynomial guarantees. Domain restrictions of the kind isolated in Section 5 remain viable.

6.3 Why Over-Specification Can Be Rational

Proposition 6.4 (Model-Conditional Rationality of Over-Specification)**.** *Consider a design process in which:*

1. *removing a parameter requires certifying that it is irrelevant,*
2. *exact irrelevance certification is charged according to the computational model of Section 2.4, and*
3. *carrying an extra parameter incurs only linear or otherwise low-order maintenance overhead.*

Then, for sufficiently expressive succinctly encoded instances, retaining extra parameters can be an economically rational response to the hardness of exact minimization.

Proof. By Theorems 3.6 and 3.7, exact sufficiency certification and exact minimization are coNP-complete in the general succinct regime. By Section 4, this hardness is accompanied by exponential lower bounds under ETH in the linear-support regime. If the cost of carrying extra parameters grows only linearly while exact minimization inherits worst-case exponential cost, then beyond a problem-dependent threshold the latter dominates the former. Under that cost model, over-specification is a rational hedge against intractable exact pruning. ■

Remark 6.5. The conclusion is not that over-specification is always best. It is that, in the absence of tractable structure, exact minimality need not be a computationally reasonable target.

6.4 Heuristics and Structural Restrictions

The tractable regimes of Section 5 explain when exact simplification is computationally realistic. When bounded action sets, separable utility, low tensor rank, tree structure, bounded treewidth, or coordinate symmetry are present, exact certification becomes feasible. Outside such regimes, approximate selection, conservative over-inclusion, and domain-specific simplification strategies are often better aligned with the complexity landscape than unconditional demands for exact minimality.

6.5 Competence by Regime

Proposition 6.6 (Exact Certification Competence Depends on Regime)**.** *Within the model of this paper, exact certification competence is regime-dependent. In the general succinct regime, exact relevance certification and exact minimization inherit the hardness results of Sections 3 and 4. In the structured regimes of Section 5, exact certification becomes available in polynomial time.*

Proof. The negative side is given by Theorems 3.6 and 3.7, together with the ETH-conditioned lower bounds summarized in Section 4. The positive side is given by the tractability results of Section 5, which provide explicit polynomial-time certification procedures under structural restrictions. ■

Remark 6.7. This is the only competence distinction needed in Paper A: not a general theory of reporting or abstention, but a theorem-driven separation between regimes in which exact certification is computationally available and regimes in which it is not.

7 Related Work

7.1 Formalized Complexity Theory

Machine verification of complexity-theoretic results remains sparse compared to other areas of mathematics. We survey existing work and position our contribution.

Coq formalizations. Forster et al. [13] developed a Coq library for computability theory, including undecidability proofs. Their work focuses on computability rather than complexity classes. Kunze et al. [21] formalized the Cook-Levin theorem in Coq, proving SAT is NP-complete. Our work extends this methodology to coNP-completeness and approximation hardness.

Isabelle/HOL. Nipkow and colleagues formalized substantial algorithm verification in Isabelle [27], but complexity-theoretic reductions are less developed. Recent work on algorithm complexity [16] provides time bounds for specific algorithms rather than hardness reductions.

Lean and Mathlib. Mathlib's computability library [38] provides primitive recursive functions and basic computability results. Our work extends this to polynomial-time reductions and complexity classes. To our knowledge, this is the first Lean 4 formalization of coNP-completeness proofs and approximation hardness.

The verification gap. Published complexity proofs occasionally contain errors [24]. Machine verification eliminates this uncertainty. Our contribution demonstrates that complexity reductions are amenable to formalization with reasonable effort (44497 lines for the full reduction suite).

7.2 Computational Decision Theory

The complexity of decision-making has been studied extensively. Papadimitriou [28] established foundational results on the complexity of game-theoretic solution concepts. Our work extends this to the meta-question of identifying relevant information. For a modern treatment of complexity classes, see Arora and Barak [3].

Closest prior work and novelty. Closest to our contribution is the literature on feature selection and model selection hardness, which proves NP-hardness of selecting informative feature subsets and inapproximability for minimum feature sets [5, 2]. Those results analyze predictive relevance or compression objectives. We study decision relevance and show coNP-completeness for sufficiency checking, a different quantifier structure (universal verification) with distinct proof techniques and a full hardness/tractability landscape under explicit encoding assumptions, mechanized in Lean 4. The formalization aspect is also novel: prior work establishes hardness on paper, while we provide machine-checked reductions with explicit polynomial bounds.

7.3 Succinct Representations and Regime Separation

Representation-sensitive complexity is established in planning and decision-process theory: classical and compactly represented MDP/planning problems exhibit sharp complexity shifts under different input models [29, 26, 25]. Our explicit-vs-succinct separation aligns with this broader principle.

The distinction in this paper is the object and scope of the classification: we classify *decision relevance* (sufficiency, anchor sufficiency, and minimum sufficient sets) for a fixed decision relation, with theorem-level regime typing and mechanized reduction chains.

7.4 Oracle and Query-Access Lower Bounds

Query-access lower bounds are a standard source of computational hardness transfer [11]. Our `[Q_fin]` and `[Q_bool]` results are in this tradition, but specialized to the same sufficiency predicate used throughout the paper: they establish access-obstruction lower bounds while keeping the underlying decision relation fixed across regimes.

7.5 Feature Selection Complexity

In machine learning, feature selection asks which input features are relevant for prediction. Blum and Langley [5] survey the field, noting hardness in general settings. Amaldi and Kann [2] proved that finding minimum feature sets for linear classifiers is NP-hard, and established inapproximability bounds: no polynomial-time algorithm approximates the minimum feature set within factor $2^{\log^{1-\epsilon} n}$ unless $\mathrm{NP} \subseteq \mathrm{DTIME}(n^{\mathrm{polylog}\, n})$.

Our results extend this line: the decision-theoretic analog (SUFFICIENCY-CHECK) is coNP-complete, and MINIMUM-SUFFICIENT-SET inherits this hardness. The key insight is that sufficiency checking is "dual" to feature selection; rather than asking which features predict a label, we ask which coordinates determine optimal action. The coNP (rather than NP) classification reflects this duality: insufficiency has short certificates (counterexample state pairs), while sufficiency requires universal verification.

Backdoors to tractable classes. Bessiere et al. [4] show how to exploit CSP instances that are "almost" tractable by computing backdoors: the smallest set of variables whose instantiation yields a tractable subproblem. Computing minimal backdoors is fixed-parameter tractable (FPT) when the tractable subset of relations is known, and W[2]-complete otherwise. This complements our dichotomy: when a decision problem has high DOF (many relevant coordinates), their backdoor method identifies the minimal intractable core. The two results together characterize the tractability boundary from different angles.

7.6 Sufficient Statistics

Fisher [12] introduced sufficient statistics: a statistic $T(X)$ is *sufficient* for parameter θ if the conditional distribution of X given $T(X)$ does not depend on θ. Lehmann and Scheffé [22] characterized minimal sufficient statistics and their uniqueness properties.

Our coordinate sufficiency is the decision-theoretic analog: a coordinate set I is sufficient if knowing s_I determines optimal action, regardless of the remaining coordinates. The parallel is precise:

- **Statistics:** T is sufficient $\iff P(X|T(X), \theta) = P(X|T(X))$
- **Decisions:** I is sufficient $\iff \mathrm{Opt}(s) = \mathrm{Opt}(s')$ whenever $s_I = s'_I$

Fisher's factorization theorem provides a characterization; our Theorem 3.7 shows that *finding* minimal sufficient statistics (in the decision-theoretic sense) is computationally hard.

7.7 Causal Inference and Adjustment Sets

Pearl [30] and Spirtes et al. [35] developed frameworks for identifying causal effects from observational data. A central question is: which variables must be adjusted for to identify a causal effect? The *adjustment criterion* and *back-door criterion* characterize sufficient adjustment sets.

Our sufficiency problem is analogous: which coordinates must be observed to determine optimal action? We conjecture that optimal adjustment set selection is intractable; recent work on the complexity of causal discovery supports this [8].

The connection runs deeper: Shpitser and Pearl [33] showed that identifying causal effects is NP-hard in general graphs. Our coNP-completeness result for SUFFICIENCY-CHECK is the decision-theoretic counterpart.

Probability of necessity and sufficiency. Cai et al. [7] formalize the Probability of Necessity and Sufficiency (PNS) for explaining Graph Neural Networks. They argue that a convincing explanation must be *both* necessary and sufficient: necessary explanations alone may be incomplete, while sufficient explanations alone may be non-concise. They model GNNs as Structural Causal Models and optimize a lower bound of PNS. This provides independent confirmation from the neural network explainability literature that necessity and sufficiency must be jointly considered—exactly the perspective underlying our sufficiency-checking framework.

7.8 Minimum Description Length and Kolmogorov Complexity

The Minimum Description Length (MDL) principle [31, 15] formalizes model selection as compression: the best model minimizes description length of data plus model. Kolmogorov complexity [23] provides the theoretical foundation (the shortest program that generates the data).

Our decision quotient connects to this perspective: a coordinate set I is sufficient if it compresses the decision problem without loss. Knowing s_I is as good as knowing s for decision purposes. The minimal sufficient set is the MDL-optimal compression of the decision problem.

The complexity results explain why MDL-based model selection uses heuristics: finding the true minimum description length is uncomputable (Kolmogorov complexity) or intractable (MDL approximations). Our results show the decision-theoretic analog is coNP-complete—intractable but decidable.

7.9 Value of Information

The value of information (VOI) framework [17] quantifies the maximum rational payment for information. Our work addresses a different question: not the *value* of information, but the *complexity* of identifying which information has value.

In explicit-state representations, VOI is polynomial to compute in the input size, while identifying which information *to value* (our problem) is coNP-complete. This separation explains why VOI is practical while optimal sensor placement remains heuristic.

7.10 Sensitivity Analysis

Sensitivity analysis asks how outputs change with inputs. Local sensitivity (derivatives) is polynomial; global sensitivity (Sobol indices [34]) requires sampling. Identifying which inputs *matter* for

decision-making is our sufficiency problem, which we show is coNP-complete.

This explains why practitioners use sampling-based sensitivity analysis rather than exact methods: exact identification of decision-relevant inputs is intractable. The dichotomy theorem (Theorem 4.1) characterizes when sensitivity analysis becomes tractable (logarithmic relevant inputs) versus intractable (linear relevant inputs).

7.11 Model Selection

Statistical model selection (AIC [1], BIC [32], cross-validation [37]) provides practical heuristics for choosing among models. Our results provide theoretical justification: optimal model selection is intractable, so heuristics are necessary. At the level of decision relevance, the lesson is structural rather than heuristic: exact identification of the smallest sufficient representation is hard in general, so restricted model classes and structural assumptions are not optional conveniences but complexity-theoretic necessities.

Three-axis integration. To our knowledge, prior work treats these pillars separately: representation-sensitive hardness, query-access lower bounds, and certifying soundness disciplines. This paper integrates all three for the same decision-relevance object in one regime-typed and machine-checked framework.

8 Conclusion

This paper isolates the abstract complexity of decision-relevant information. Once the state space is factored into coordinates, the central question is not merely how to evaluate the objective on a given state, but how to certify which coordinates determine the optimal action.

Main Results

Within the encoding model of Section 2.4, we establish:

- Sufficiency-Check is coNP-complete
- Minimum-Sufficient-Set is coNP-complete
- Anchor-Sufficiency is Σ_2^{P}-complete
- an explicit-state versus succinct dichotomy under which logarithmic-size sufficient sets admit polynomial-time algorithms while linear-size sufficient sets inherit ETH-conditioned exponential lower bounds
- tractable subcases under bounded actions, separable utility, low tensor rank, tree structure, bounded treewidth, and coordinate symmetry

Taken together, these results identify a precise source of difficulty: certifying that information may be discarded can be harder than evaluating the full decision rule once all information is already present.

Engineering Consequences

The engineering consequences are direct corollaries of the main theorems. Configuration simplification is an instance of sufficiency checking; exact minimization has no general-purpose polynomial-time worst-case solution unless $\mathsf{P} = \mathsf{coNP}$; and in the hard succinct regime, conservative over-specification can be a rational response when maintenance overhead is low-order and exact pruning cost is exponential.

Mechanization

The Lean 4 artifact machine-checks the main reduction constructions, hardness transfers, and tractability statements. The mechanization does not replace the mathematical argument; it certifies that the formal definitions and reductions used by the argument are internally correct and reproducible.

Outlook

Two immediate directions remain. First, the reduction infrastructure can be further integrated with general-purpose formalized complexity libraries. Second, the abstract theorem stack developed here can serve as a clean upstream citation target for downstream convergence and physical papers without forcing those themes into the foundations-and-complexity presentation.

A Lean 4 Proof Listings

The complete Lean 4 formalization is available at:

https://doi.org/10.5281/zenodo.18140966

The formalization is not a transcription of the paper text. It exposes reusable definitions for reductions, decision problems, hardness transfers, and tractable subcases, so the main complexity claims can be checked mechanically.

A.1 What the Artifact Covers

The appendix supports the non-physical theorem stack of the paper:

- core decision-problem and sufficiency definitions
- polynomial-time reduction infrastructure
- coNP and Σ_2^P hardness reductions
- ETH-conditioned dichotomy statements
- tractable subcases under explicit structural assumptions

The point of the artifact is precision. Once the definitions are fixed, Lean verifies that the reductions preserve the intended predicates and that the later theorems invoke the correct formal statements.

A.2 Module Structure

The formalization consists of 179 files organized as follows:

- `Basic.lean` and `Sufficiency.lean` – core definitions for decision problems, projections, optimizer maps, and sufficiency
- `AlgorithmComplexity.lean` and `PolynomialReduction.lean` – polynomial-time reduction infrastructure and composition
- `Reduction.lean` and `Hardness/` – the TAUTOLOGY, SET-COVER, ETH, and Σ_2^P reduction stack
- `QueryComplexity.lean`, `Dichotomy.lean`, and `ComplexityMain.lean` – query lower bounds and summary complexity statements
- `Tractability/` – bounded actions, separable utility, tensor-rank, tree-structured, treewidth, and symmetry arguments

A.3 Key Theorems

Theorem A.1 (Polynomial Composition, Lean)**.** *Polynomial-time reductions compose to polynomial-time reductions.*

```
theorem PolyReduction.comp_exists
    (f : PolyReduction A B) (g : PolyReduction B C) :
    exists h : PolyReduction A C,
      forall a, h.reduce a = g.reduce (f.reduce a)
```

Theorem A.2 (TAUTOLOGY Reduction Correctness, Lean)**.** *The canonical reduction used in Theorem 3.6 is mechanized and proves that the empty set is sufficient exactly when the source formula is a tautology.*

```
theorem reduction_correct
    (φ : QBF) :
    empty_sufficient (reduce φ) ↔ QBF.eval φ
```

A.4 Verification Status

- Total lines: 44497
- Theorems/lemmas: 1872
- Files: 179
- Status: All proofs compile with 0 `sorry`

References

[1] Hirotugu Akaike. A new look at the statistical model identification. *IEEE Transactions on Automatic Control*, 19(6):716–723, 1974.

[2] Edoardo Amaldi and Viggo Kann. On the approximability of minimizing nonzero variables or unsatisfied relations in linear systems. *Theoretical Computer Science*, 209(1-2):237–260, 1998.

[3] Sanjeev Arora and Boaz Barak. *Computational Complexity: A Modern Approach*. Cambridge University Press, 2009.

[4] Christian Bessiere, Clement Carbonnel, Emmanuel Hebrard, George Katsirelos, and Toby Walsh. Detecting and exploiting subproblem tractability. In *Proceedings of the Twenty-Third International Joint Conference on Artificial Intelligence (IJCAI)*, pages 468–474, 2013.

[5] Avrim L. Blum and Pat Langley. Selection of relevant features and examples in machine learning. *Artificial Intelligence*, 97(1-2):245–271, 1997.

[6] Hans L. Bodlaender. A tourist guide through treewidth. In *Acta Cybernetica*, volume 11, pages 1–21, 1993.

[7] Ruichu Cai, Yuxuan Zhu, Xuexin Chen, Yuan Fang, Min Wu, Jie Qiao, and Zhifeng Hao. On the probability of necessity and sufficiency of explaining graph neural networks: A lower bound optimization approach. *Neural Networks*, 184:107065, 2025.

[8] David Maxwell Chickering, David Heckerman, and Christopher Meek. Large-sample learning of Bayesian networks is NP-hard. In *Journal of Machine Learning Research*, volume 5, pages 1287–1330, 2004.

[9] Andrzej Cichocki, Namgil Lee, Ivan Oseledets, Anh-Huy Phan, Qibin Zhao, and Danilo P. Mandic. *Tensor Networks for Dimensionality Reduction and Large-Scale Optimization*, volume 9. Now Publishers, 2016.

[10] Stephen A. Cook. The complexity of theorem-proving procedures. In *Proceedings of the Third Annual ACM Symposium on Theory of Computing*, pages 151–158. ACM, 1971.

[11] Shahar Dobzinski and Jan Vondrák. From query complexity to computational complexity. In *Proceedings of the 44th ACM Symposium on Theory of Computing*, pages 1107–1116, 2012.

[12] Ronald A. Fisher. On the mathematical foundations of theoretical statistics. *Philosophical Transactions of the Royal Society of London. Series A*, 222:309–368, 1922.

[13] Yannick Forster, Edith Heiter, and Gert Smolka. Verified programming of Turing machines in Coq. In *Proceedings of the 8th ACM SIGPLAN International Conference on Certified Programs and Proofs*, pages 114–127. ACM, 2019.

[14] Eugene C. Freuder. Complexity of k-tree structured constraint satisfaction problems. *Proceedings of AAAI*, pages 4–9, 1990.

[15] Peter D. Grünwald. *The Minimum Description Length Principle*. MIT Press, 2007.

[16] Maximilian P. L. Haslbeck and Tobias Nipkow. Verified analysis of list update algorithms. In *Journal of Automated Reasoning*, volume 65, pages 117–156. Springer, 2021.

[17] Ronald A. Howard. Information value theory. *IEEE Transactions on Systems Science and Cybernetics*, 2(1):22–26, 1966.

[18] Russell Impagliazzo and Ramamohan Paturi. On the complexity of k-sat. *Journal of Computer and System Sciences*, 62(2):367–375, 2001.

[19] Richard M. Karp. Reducibility among combinatorial problems. In *Complexity of Computer Computations*, pages 85–103. Springer, 1972.

[20] Tamara G. Kolda and Brett W. Bader. Tensor decompositions and applications. *SIAM Review*, 51(3):455–500, 2009.

[21] Fabian Kunze, Gert Smolka, and Yannick Forster. Formal small-step verification of a call-by-value lambda calculus machine. In *Asian Symposium on Programming Languages and Systems*, pages 264–283. Springer, 2019.

[22] Erich L. Lehmann and Henry Scheffé. Completeness, similar regions, and unbiased estimation. *Sankhyā: The Indian Journal of Statistics*, 10(4):305–340, 1950.

[23] Ming Li and Paul Vitányi. *An Introduction to Kolmogorov Complexity and Its Applications*. Springer, 3rd edition, 2008.

[24] Richard J. Lipton. The P=NP question and Gödel's lost letter. *Communications of the ACM*, 52(5):31–33, 2009.

[25] Michael L. Littman, Judy Goldsmith, and Martin Mundhenk. The computational complexity of probabilistic planning. *Journal of Artificial Intelligence Research*, 9:1–36, 1998.

[26] Martin Mundhenk, Judy Goldsmith, Christopher Lusena, and Eric Allender. The complexity of finite-horizon markov decision process problems. *Journal of the ACM*, 47(4):681–720, 2000.

[27] Tobias Nipkow, Lawrence C. Paulson, and Markus Wenzel. *Isabelle/HOL: A Proof Assistant for Higher-Order Logic*. Springer, 2002.

[28] Christos H. Papadimitriou. *Computational Complexity*. Addison-Wesley, 1994.

[29] Christos H. Papadimitriou and John N. Tsitsiklis. The complexity of markov decision processes. *Mathematics of Operations Research*, 12(3):441–450, 1987.

[30] Judea Pearl. *Causality: Models, Reasoning, and Inference*. Cambridge University Press, 2nd edition, 2009.

[31] Jorma Rissanen. Modeling by shortest data description. *Automatica*, 14(5):465–471, 1978.

[32] Gideon Schwarz. Estimating the dimension of a model. *The Annals of Statistics*, 6(2):461–464, 1978.

[33] Ilya Shpitser and Judea Pearl. Identification of joint interventional distributions in recursive semi-Markovian causal models. *Proceedings of the National Conference on Artificial Intelligence*, 21(2):1219–1226, 2006.

[34] Ilya M. Sobol. Global sensitivity indices for nonlinear mathematical models and their Monte Carlo estimates. *Mathematics and Computers in Simulation*, 55(1-3):271–280, 2001.

[35] Peter Spirtes, Clark Glymour, and Richard Scheines. *Causation, Prediction, and Search*. MIT Press, 2nd edition, 2000.

[36] Larry J. Stockmeyer. The polynomial-time hierarchy. *Theoretical Computer Science*, 3(1):1–22, 1976.

[37] Mervyn Stone. Cross-validatory choice and assessment of statistical predictions. *Journal of the Royal Statistical Society: Series B (Methodological)*, 36(2):111–133, 1974.

[38] The mathlib Community. The Lean mathematical library. *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*, pages 367–381, 2020.